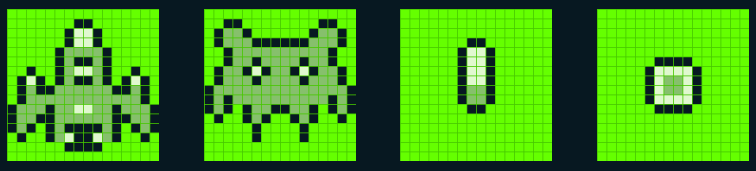
GB STUDIO
FOR ABSOLUTE BEGINNERS

A slow, simple, step-by-step guide to making your first Game Boy game,
ending with a **working vertical shoot-’em-up** called **STAR PATROL**





No programming knowledge needed · Written for GB Studio 4

Before you start

Welcome! This guide will take you from **“I have never made a game”** to **“I made a real Game Boy game that I can play and share”**. We will go slowly. Every click is written down. Nothing is skipped.

The guide has three parts:

Part	What you will do	Time
Part 1 – Learn the basics	Install GB Studio, look around the editor, and understand the few ideas that every GB Studio game is built from.	about 45 min
Part 2 – Build STAR PATROL	Build a complete vertical shoot-’em-up: title screen, a scrolling space level, shooting, enemies, score, lives, Game Over and Victory screens.	about 2 hours
Part 3 – Make it yours	Bonus features (enemy bullets, new enemy patterns), a troubleshooting table, and a cheat sheet with every script in one place.	as long as you like

What you need

- A computer with Windows, macOS or Linux.
- **GB Studio 4** (free – Chapter 2 shows you how to get it).
- The **art files** that come with this guide: four sprites and four backgrounds (PNG pictures). They are already made in the correct Game Boy format, so you do not need to draw anything. If you want to draw your own later, Chapter 5 explains the rules.
- Patience. Read one step, do one step, then read the next.

How to read this guide

Numbered steps look like this – do them in order:

1. Read the step.
2. Do exactly what it says.
3. Only then move to the next step.

You will also see coloured boxes:

> **TIP**
> Little shortcuts and helpful ideas.

> **WHY DO WE DO THIS?**
> An explanation of *why* we do something. Understanding the “why” is what turns you into a game maker, not just someone following instructions.

WATCH OUT
A common mistake. Read these carefully!

CHECKPOINT – TEST IT!
A moment to press Play and test your game. Testing often is the most important habit in game making. If something is wrong, you will find it right away, while it is still easy to fix.

NOTE
This guide was written for **GB Studio 4**. GB Studio is updated often, so a button may sit in a slightly different place or an event may have a slightly different name in your version. Don’t worry: when you add an event, a **search box** appears – just type a keyword such as *projectile* or *scene* and the right event will show up. The official documentation at **gbstudio.dev/docs** always matches the newest version.

PART 1

Learn the basics

Chapters 1–7: what GB Studio is, how to install it, a tour of the editor, and the few big ideas you need.

Chapter 1 – What is GB Studio?

GB Studio is a free program that lets you make games for the original Nintendo **Game Boy** (and the Game Boy Color) **without writing code**. Instead of typing programming commands, you build your game by:

- **placing things** (backgrounds, characters, invisible zones) on a map with your mouse, and
- **choosing “events”** from menus – little instructions like “play a sound”, “move this enemy”, “go to the next screen”.

When you are done, GB Studio turns your project into a real Game Boy **ROM file** (a file ending in **.gb**). That file runs in any Game Boy emulator, on real Game Boy hardware with a special cartridge, and GB Studio can also export a **web version** that runs in a browser, ready to upload to websites like itch.io.

WHY DO WE DO THIS?
Why make games for a console from 1989? Because its limits make it **perfect for learning**. The screen is small, there are only four colours, and there is only so much you can fit in. You cannot get lost in a giant project. You finish games – and finishing is the skill that matters most.

What kinds of games can you make?

GB Studio has built-in “scene types” for many genres: top-down adventures (like classic Zelda or Pokémon), platformers (like Mario), point-and-click adventures – and the one we want: **Shoot ’Em Up**, a scrolling space shooter. You will learn more about scene types in Chapter 7.

Chapter 2 – Installing GB Studio

1. Open your web browser and go to **gbstudio.dev** (the official site). Click **Download**. You will be taken to GB Studio’s page on **itch.io** (chrismaltby.itch.io/gb-studio).
2. On the itch.io page click **Download Now**. GB Studio is free; itch.io may ask if you want to pay something to support the creator. You can choose “No thanks, just take me to the downloads” – or support the project if you like it.
3. Download the version for your system: **Windows**, **macOS** or **Linux**.
4. **Windows:** unzip the downloaded file and run the installer. **macOS:** unzip and drag *GB Studio* into your *Applications* folder. **Linux:** use the .deb package or the AppImage.
5. Start GB Studio. You will see a small **welcome window** with options such as **New**, **Open** and **Recent**. If you see that window – congratulations, it works!

WATCH OUT

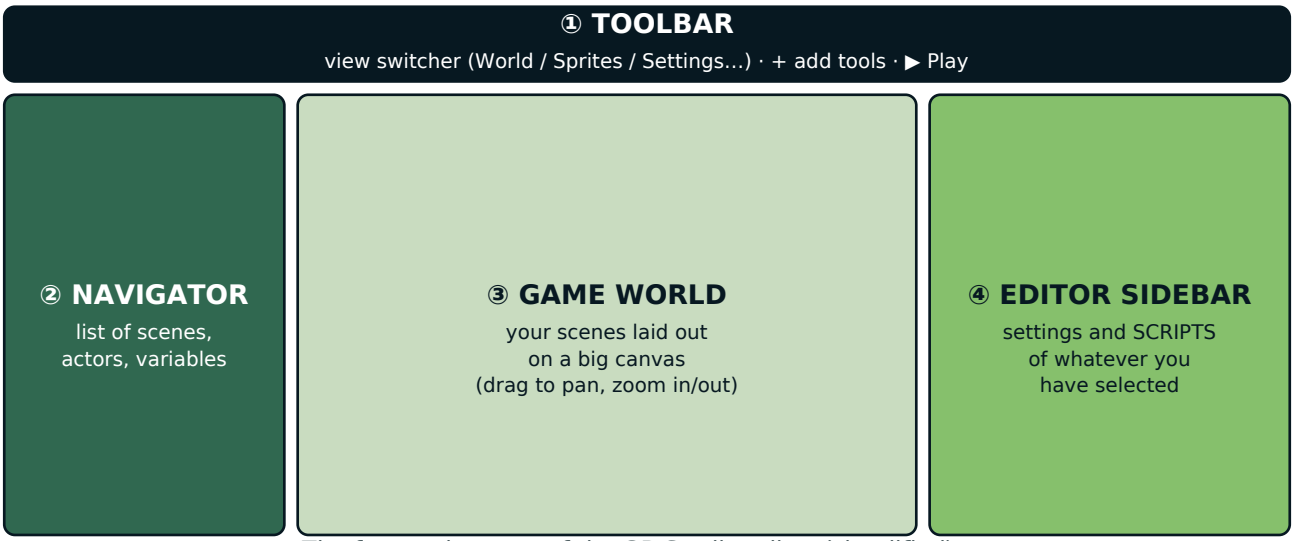
On macOS, the first time you open GB Studio you might see a warning that the app is from the internet. Right-click (or Control-click) the app and choose **Open**, then confirm. On Windows, SmartScreen may show “Windows protected your PC” – click **More info → Run anyway**. Only do this for files you downloaded from the official page.

Chapter 3 – Your first look at the editor

Let’s create a practice project just to look around. (We will make the real Star Patrol project in Part 2.)

1. In the welcome window click **New**.
2. Give it the name **Practice**, pick a folder you can find again (for example *Documents*), and choose the **Sample Project** template. The sample is a small finished game – perfect for exploring.
3. Click **Create**. The main editor window opens.

The editor has four main areas:



The four main areas of the GB Studio editor (simplified).

Area	What it is for
① **Toolbar** (top)	Switch between views (Game World, Sprites, Backgrounds, Music, Palettes, Settings and so on), the **+** button to add scenes/actors/triggers, zoom, and the ▶ **Play** button that runs your game.
② **Navigator** (left)	A list of everything in the project: scenes, the actors and triggers inside them, and your **variables**. Click any name to jump to it.
③ **Game World** (middle)	A big canvas showing all your scenes. Drag the empty grey area to move around; zoom with the toolbar buttons or the mouse wheel together with Ctrl (Cmd on Mac).
④ **Editor Sidebar** (right)	Shows the settings and the **scripts** of whatever you selected: a scene, an actor, a trigger, or the project itself. You will spend a lot of time here.

Try it: play the sample game

1. Click the ▶ **Play** button in the toolbar (top right).
2. GB Studio “builds” the game. This takes a few seconds the first time. A new window opens with a Game Boy screen.
3. Play! The default keyboard controls are: **arrow keys** = D-pad, **Z** or **J** = A button, **X** or **K** = B button, **Enter** = Start, **Shift** = Select.
4. Close the game window when you are done.

TIP
Play the game often while you build. The Play button is your best friend. The keys above are the defaults; you can see and change them in GB Studio’s settings/preferences.

Try it: click around

1. In the Game World, click once on the **name/title bar** above any scene. The sidebar now shows that **scene**: its name, type, background and its scripts.
2. Click on a **character** standing in a scene. The sidebar now shows that **actor**.
3. Click on an empty grey area outside all scenes. The sidebar now shows **project** settings.
4. Don’t change anything yet – just notice how the sidebar always shows the thing you selected.

WHY DO WE DO THIS?
This “select something → edit it in the sidebar” pattern is how **everything** works in GB Studio. Once you understand it, the whole program makes sense.

Chapter 4 – The building blocks of every game

Every GB Studio game – from the simplest to the most complex – is made from the same handful of pieces. Learn these words and you will understand any tutorial.

Word	Simple meaning	In STAR PATROL
Scene	One “place” or “screen” of your game. Like a room in a house, or a page in a book. The game is always in exactly one scene.	Title screen, Level 1, Game Over, Victory
Background	The picture painted behind everything in a scene. It doesn’t move by itself.	Starry space, the title picture
Sprite (sprite sheet)	A small picture (usually 16×16 pixels) for something that moves or can be animated.	The ship, the alien, the bullets
Actor	A “thing” placed in a scene that uses a sprite and can have scripts. People, enemies, coins, doors…	Each alien enemy
Player	The special actor that *you* control with the D-pad. Every scene (except Logo scenes) has one.	Your space ship
Trigger	An *invisible* rectangle on the map. When the player enters it, its script runs.	The FINISH line
Projectile	A short-lived flying sprite launched by a script – a bullet, arrow or fireball.	Laser shots
Event	One instruction, chosen from a menu: “Play sound”, “Change scene”, “Hide actor”…	Dozens of them
Script	A list of events that run one after another, from top to bottom.	“When an alien is hit: play a sound, add 1 to score, remove the alien”
Variable	A named box that stores a number, so the game can remember things.	**Score** and **Lives**
Scene type	Decides how the player moves in a scene (walking, jumping, flying…).	Shoot ’Em Up for the level

TIP

A good way to remember: **Scenes** are the stage, **backgrounds** are the painted scenery, **actors** are the performers, **sprites** are their costumes, and **scripts** are what the performers are told to do.

Chapter 5 – Game Boy rules you must know

The Game Boy is an old, small machine. GB Studio has a few rules because of this. If you know them, your art will import without errors.

Rule 1: the screen is tiny

The Game Boy screen is **160 pixels wide and 144 pixels tall**. A scene can be bigger than the screen – then the camera scrolls around it – but you only ever *see* 160×144 at a time.

Rule 2: everything is made of 8×8 tiles

The Game Boy draws pictures in small squares of **8×8 pixels** called **tiles**. A full screen is **20 tiles wide and 18 tiles tall**. Look at our title screen with the tile grid drawn on top:



The title screen split into 8×8 tiles: 20 columns × 18 rows.

Two important consequences:

- Background pictures must have a width and height that are **multiples of 8** (160, 168, 176… / 144, 152, 160…).
- Positions in GB Studio are usually measured **in tiles**, not pixels. “X = 9, Y = 156” means 9 tiles from the left and 156 tiles from the top.

Rule 3: only four colours

The original Game Boy shows only **four shades of green**. In your PNG files you must use exactly these colours:



#071821 Darkest	#306850 Dark	#86c06c Light	#e0f8cf Lightest	#65ff00 Transparent
sprites ✓ bg ✓	bg only ✗ sprites	sprites ✓ bg ✓	sprites ✓ bg ✓	sprites only

The official GB Studio palette. Backgrounds use the four greens; sprites use three greens plus bright green for “transparent”.

- **Backgrounds** use the four colours #071821, #306850, #86c06c, #e0f8cf.
- **Sprites** use #071821, #86c06c, #e0f8cf – and the bright green **#65ff00** means “see-through”. The colour #306850 cannot be used in sprites.
- Colours that are not exactly these will be changed to the nearest one, which can make your art look wrong.

Rule 4: sprites are 16×16 pixels

A simple sprite is a **16×16** pixel PNG. Animated sprites put several 16×16 frames side by side (a 2-frame animation is 32×16). In this guide every sprite is one single 16×16 frame, to keep things simple.

Rule 5: limits per scene

Limit	Value	What it means for us
Actors per scene	20 maximum	We will use 15 enemies – safe.
Actors visible on one screen	about 10	More than 10 near each other will start to flicker or disappear. Spread enemies out.
Triggers per scene	30 maximum	We use just 1.
Unique background tiles	about 128–192 (monochrome)	Big detailed pictures can be too complex. Repeated tiles (like empty space) are free. Our level is 160 tiles tall but uses only 50 *different* tiles.

TIP
Under every scene in the Game World you can see a small bar like **A: 3/20 S: 12/96 T: 1/30**. It shows how many Actors, Sprite tiles and Triggers the scene uses compared with the maximum. Keep an eye on it.

NOTE
You don’t need any drawing program for this guide – all eight pictures are supplied. Later, a pixel art editor like **Aseprite**, **LibreSprite** or the free web tool **Piskel** works great. GB Studio’s website offers ready palette files for them.

Chapter 6 – How scripts work

Scripts are the “brain” of your game. A script is simply **a list of events that run from top to bottom**, one after another, like a recipe.

When does a script run?

Every scene, actor and trigger has several **script tabs** in the sidebar. Each tab runs at a different moment:

Script tab	Belongs to	Runs when…
On Init	Scene or actor	…the scene starts (once). Great for setup: set variables, start music, set up buttons.
On Update	Actor	…over and over, in a loop, while the actor is on screen. Great for movement patterns.
On Hit	Actor (needs a Collision Group)	…this actor is touched by the player or hit by a projectile. Has sub-tabs: **Player**, **Group 1**, **Group 2**, **Group 3** – one for each kind of thing that can hit it.
On Interact	Actor	…the player presses A next to it (top-down games). In Shoot ’Em Up scenes, touching counts as interacting.
On Enter / **On Leave**	Trigger	…the player walks into / out of the trigger’s rectangle.
On Player Hit	Scene	…the player is hit by a projectile of a certain group (used in Bonus A).

How to add an event – the basic skill

1. Select the thing (scene, actor or trigger).
2. In the sidebar, click the **script tab** you want (for example **On Init**).
3. Click the **+ Add Event** button.
4. A menu appears with a **search box**. Type a word (e.g. *sound*) and click the event you want.
5. The event appears in the script as a box with some fields. Fill them in.
6. To add the next event, click **+ Add Event** again *below* the first one.

> **TIP**
> You can drag events up and down to change their order, and each event box has a small menu (often a ▾ or ⋮ button) with options like **Delete**, **Copy** and **Disable**. Some events, like *If…* or *Attach Script To Button*, contain their **own** little script inside them – you add events inside using the + Add Event button that appears inside that box.

How scripts are shown in this guide

Whenever you must build a script, this guide shows it as a block like the one below. The dark header tells you **where** the script goes. Each white card is **one event**, in order, with the fields you must set. Cards that are shifted to the right (with an orange edge) go **inside** the event above them.

EXAMPLE · Some Actor → On Hit → Player

Play Sound Effect
Sound: **Crash**

If Variable Compare With Value
Variable: **Lives** · Comparison: **≤** · Value: **0**

> **Change Scene (inside the “If” – in its TRUE part)**
> Scene: **Game Over**

Read it as: “When the player touches this actor, play a crash sound. Then, *if* lives are 0 or less, go to the Game Over scene.”

Three kinds of events you will use all the time

- **Doing things**: Play Sound Effect, Change Scene, Launch Projectile, Hide Actor, Move Relative…
- **Remembering things** (variables): Set To Value, Increment By 1, Decrement By 1…
- **Deciding things** (control flow): *If Variable Compare With Value* runs one set of events when something is true, and (optionally) another when it is false.

Chapter 7 – Scene types

Each scene has a **Type**. The type decides how the player moves and how collisions work.

Scene type	Player moves like…	Good for
Top Down 2D	Step by step on a grid, in 4 directions	RPGs, menus, simple screens
Adventure	Smoothly in all directions	Action adventures
Platformer	Runs and jumps with gravity	Mario-style games
Point and Click	As a mouse-like cursor	Adventure games with clicking
Logo	There is no player at all	Title screens, splash pictures (allows more detailed backgrounds)
Shoot ’Em Up	Flies freely while the screen **scrolls by itself**	**Space shooters – our game!**

WHY DO WE DO THIS?

The most important fact about the Shoot ’Em Up scene type: **the scrolling direction is decided by the direction the player is FACING when the scene starts.**

Player facing **right** → horizontal shooter (screen scrolls right). Player facing **up** → **vertical shooter** (screen scrolls up). So to make a vertical shooter, we only have to make sure the ship faces **Up** when it enters the level. Remember this – it is the key to the whole project!

Now you know enough. Close the practice project (or keep it for experimenting) and let’s build a real game!

PART 2

Build STAR PATROL

Steps 0–13: from an empty project to a finished, exported vertical shoot-’em-up.

Step 0 – The plan

Professional game makers always start with a small plan. Here is ours. Read it once so you know where we are going.

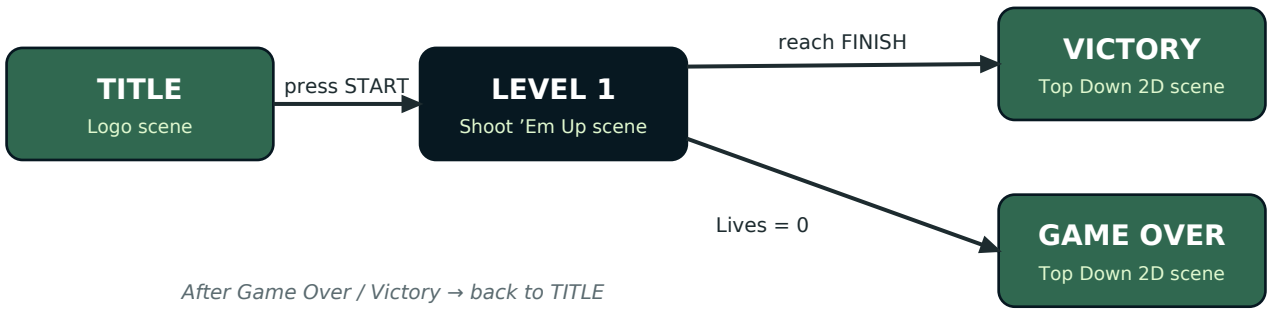
The game in one sentence

“Fly your ship up through space, shoot the aliens for points, don’t crash into them, and reach the FINISH line at the top.”

The rules

- You start with **3 lives** and **0 points**.
- The screen scrolls **upwards** automatically. You can fly anywhere on the screen with the D-pad.
- Press **A** to shoot a laser straight up.
- Every alien you shoot = **+1 point**.
- Touching an alien = **–1 life** (and the alien explodes).
- Lives reach 0 → **GAME OVER** screen. Reach the finish line → **MISSION COMPLETE** screen. Both show your score and lead back to the title.

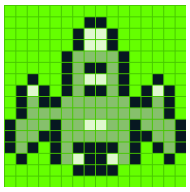
The four scenes



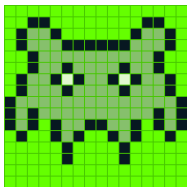
How the four scenes connect.

The art files

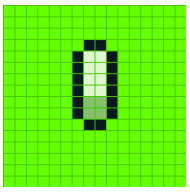
These eight files come with this guide (in the folder **StarPatrol_assets**):



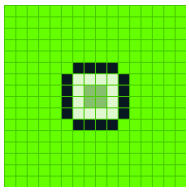
player_ship.png
your ship, 16×16



enemy.png
alien, 16×16



bullet.png
your laser, 16×16



enemy_bullet.png
alien shot (bonus)

The bright green squares are the **transparent** colour – they will be invisible in the game.



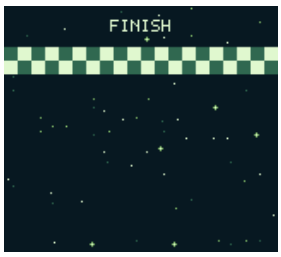
title.png
160×144



gameover.png
160×144



victory.png
160×144



level1.png
160×1280 (8 screens tall!) – top part shown

Step 1 – Create the project and add the art

1A. Create the project

1. Start GB Studio. In the welcome window click **New**.
2. Name: **StarPatrol** (no spaces is safest).
3. Path/folder: choose somewhere easy, e.g. **Documents/GBStudio**. Remember it – you’ll need it in a minute.
4. Template: choose the **empty / blank** project template (named *Empty* or similar). We want a clean start, without the sample game’s scenes.
5. Click **Create**. The editor opens. The Game World may be completely empty, or have one empty scene.

> **NOTE**
> If your empty template still contains a scene, that’s fine – we will reuse or delete it. To delete a scene: select it and press **Delete/Backspace**, or use the small menu (▾/ ⋮) at the top of the sidebar and choose **Delete Scene**.

1B. Copy the art into the project

GB Studio does not have its own drawing tools for backgrounds. Instead, it watches special folders inside your project. Any PNG you put there appears in GB Studio automatically.

1. Open your computer’s file manager (Explorer on Windows, Finder on Mac).
2. Go to the project folder you chose, e.g. **Documents/GBStudio/StarPatrol**. Inside you will see a file called **StarPatrol.gbsproj** and a folder called **assets**.
3. Open **assets**. Inside are folders such as **backgrounds**, **sprites**, **music**, **ui**…
4. Copy **player_ship.png**, **enemy.png**, **bullet.png** and **enemy_bullet.png** into **assets/sprites**.
5. Copy **title.png**, **level1.png**, **gameover.png** and **victory.png** into **assets/backgrounds**.
6. Switch back to GB Studio. Use the **view switcher** in the toolbar (it usually says *Game World*) and choose **Sprites**. You should see your four sprites in the list. Then choose **Backgrounds** – you should see the four backgrounds.
7. Switch back to **Game World**.

WATCH OUT

If the **sprites** or **backgrounds** folder does not exist, create it yourself with exactly that name (all lower-case). Put the files directly in it, not inside another sub-folder you made by accident.

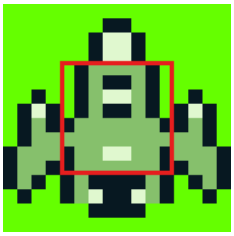
TIP

Save often with **Ctrl+S** (Cmd+S on Mac). GB Studio also autosaves in many situations, but a habit of saving never hurts.

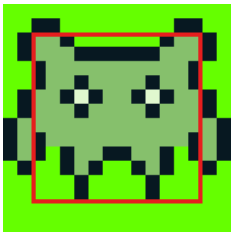
Step 2 – Set the sprites’ collision boxes

Every sprite has an invisible **collision box** (bounding box). The game uses this box – not the picture – to decide if two things touch. If the box is wrong, bullets will miss aliens that they visually hit, or you will lose lives without touching anything.

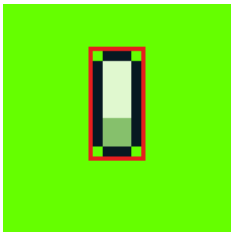
1. Switch to the **Sprites** view (toolbar view switcher).
2. Click **player_ship** in the list on the left.
3. In the sidebar on the right, find **Collision Bounding Box** (with X, Y, Width, Height). A coloured rectangle is drawn over the sprite in the middle of the screen.
4. Change the numbers until the rectangle looks like the red box in the pictures below. Suggested values are given, but **trust your eyes**: what matters is how the box looks over the picture.
5. Repeat for **enemy**, **bullet** and **enemy_bullet**.



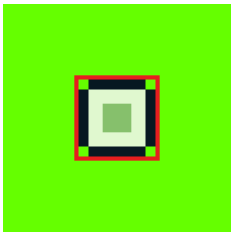
player_ship
small box in the centre
≈ X 4, Y 4, W 8, H 8



enemy
covers the body
≈ X 2, Y 2, W 12, H 12



bullet
just the laser
≈ X 6, Y 3, W 4, H 8



enemy_bullet
just the ball
≈ X 5, Y 5, W 6, H 6

WHY DO WE DO THIS?

Why is the player’s box so small? It’s a classic shoot-’em-up trick! A small “hitbox” on the player feels **fair**: when an alien only touches your wing tip, you survive. Players love that. Enemies get a big box so that your shots hit them easily.

NOTE

The exact meaning of X and Y (measured from the top-left corner or from another anchor point) can differ between GB Studio versions. That’s why the pictures matter more than the numbers – just make the drawn box match.

Step 3 – Create the variables

We need two “memory boxes”: **Score** and **Lives**. GB Studio already has many variables, called *Variable 0*, *Variable 1*… (or $00, $01…). We will simply give two of them better names.

1. In the **Navigator** on the left, find the **Variables** section and open it.
2. Click the first variable (*Variable 0* / $00). In the sidebar, type the new name **Score**.
3. Click the next one (*Variable 1* / $01) and rename it **Lives**.

TIP
You can also rename a variable later, directly inside any event that uses it: the variable dropdown usually has a small pencil/rename option. Good names make scripts much easier to read.

WHY DO WE DO THIS?
Why not just remember the score “in the game”? Because a game is just a computer program – it only remembers what you tell it to store. Variables are how your game remembers anything: score, lives, keys collected, whether a door is open…

Step 4 – The Title scene

4A. Create the scene

1. Make sure you are in the **Game World** view.
2. Click the **+** button in the toolbar and choose **Scene** (shortcut: press **S**). Then click on an empty area of the Game World. A new scene appears.
3. With the new scene selected, look at the sidebar. Set the **name** (at the top) to **Title**.
4. Set **Type** to **Logo**.
5. Set **Background** to **title**. The STAR PATROL picture appears in the scene.

WHY DO WE DO THIS?
Why the Logo type? A title screen needs no player ship flying around. Logo scenes have no player and they allow more detailed backgrounds – our title picture uses about 100 different tiles, so Logo is the safe choice.

4B. Make sure the game starts here

The game must begin on the Title scene. The first scene you create normally becomes the **starting scene** automatically. To check:

1. Click an **empty grey area** of the Game World (not on a scene). The sidebar shows the **project** settings.
2. Find **Starting Scene** (it may be in the sidebar or in the Settings view) and make sure it says **Title**. If not, choose Title.

4C. The title script (part 1)

Click the Title scene’s name bar to select it, open the **On Init** tab, and build this script:

Title (scene) → On Init

Variable Set To Value
Variable: **Score** · Value: **0**

Variable Set To Value
Variable: **Lives** · Value: **3**

Await Input
Buttons: **Start** and **A** (tick both)

Play Sound Effect
Effect: **Beep** (or any sound you like)

In words: when the title appears, reset the score to 0 and lives to 3, wait until the player presses Start or A, then beep. (We’ll add “go to the level” in Step 5.)

WHY DO WE DO THIS?

Why reset Score and Lives *here*? Because after Game Over the player comes back to this title screen. If we didn’t reset them, the second game would start with 0 lives and the old score!

CHECKPOINT – TEST IT!

Press ▶ **Play**. You should see the STAR PATROL title screen. Press **Enter** (Start) or **Z** (A) – you should hear a beep. Nothing else happens yet. That’s correct!

Step 5 – Level 1: make the world scroll

5A. Create the level scene

1. Click **+** → **Scene** and click an empty spot **to the right** of the Title scene.
2. Name it **Level 1**.
3. Type: **Shoot ’Em Up**.
4. Background: **level1**. The scene becomes very tall – 1280 pixels, eight screens high! Zoom out (toolbar zoom buttons, or Ctrl/Cmd + mouse wheel) to see all of it. The FINISH line is at the very top.

5B. Choose the player’s sprite and movement settings

The player needs to look like a space ship in this scene. GB Studio lets you set a **default player sprite for each scene type**, and also override it per scene.

1. Open the **Settings** view (toolbar view switcher → **Settings**).
2. Find the section about **scene types** (in GB Studio 4 it lists each type with its own settings). Go to **Shoot ’Em Up**.
3. Set its **Player Sprite Sheet** to **player_ship**.
4. Find the player movement option and choose **free movement** – so the ship can fly around the whole screen, not only along one edge.
5. Find the **scroll speed** setting for Shoot ’Em Up and leave it at the default for now. You can change it later to make the game easier or harder.
6. Go back to the **Game World**.

> **TIP**
> Alternatively, select the Level 1 scene: its sidebar has a **Player Sprite Sheet** option that overrides the default for this one scene only. Either way works.

> **NOTE**
> GB Studio also wants a player sprite for the **Top Down 2D** type (used later for Game Over and Victory). If the Top Down default is empty, set it to **player_ship** too – we will hide the player in those scenes anyway.

5C. Send the player to the level – facing UP!

Now go back to the **Title** scene’s **On Init** script and add one more event at the very bottom:

Title (scene) → On Init · add at the end

Change Scene
Scene: **Level 1**
Position: **X 9**, **Y 156** (you can also click the target icon and click the spot in the level)
Direction: **Up** ← the most important setting in this guide!
Fade: default

WHY DO WE DO THIS?

X 9 puts the ship near the middle (the level is 20 tiles wide). **Y 156** puts it near the bottom (the level is 160 tiles tall). So the ship starts at the bottom and flies up toward the FINISH line.

Direction: Up – remember Chapter 7: in a Shoot ’Em Up scene, *the direction the player faces when entering decides the scroll direction*. Up = vertical shooter. If you chose Right, you would get a horizontal shooter instead!

CHECKPOINT – TEST IT!

Press ▶ **Play**. Press Start on the title screen.

You should now see your ship near the bottom of the screen, and the stars **sliding downwards** – that means the camera is flying upwards. Use the arrow keys to fly around. After a while you reach the FINISH line at the top and the scrolling stops.

Stars moving sideways? → the Direction in Change Scene is not Up. **No ship visible?** → check the player sprite (5B). **Ship only moves left/right?** → choose free movement in Settings (5B).

TIP

Each test now starts at the title screen. That’s fine – just press Enter. Testing the full path every time also tests that your scenes connect correctly.

Step 6 – Shooting!

A shooter needs shots. We will tell the game: “*whenever the A button is pressed in this scene, launch a laser from the player, straight up*”. The event that does this is **Attach Script To Button**, and we add it to the Level 1 scene’s **On Init** so it is set up as soon as the level starts.

1. Select the **Level 1** scene (click its name bar).
2. Open its **On Init** tab.
3. Add the event **Attach Script To Button** (search: *attach*).
4. Set Button to **A**. If there is an option **Override default button action**, tick it.
5. **Inside** this event there is its own “+ Add Event” button. Use it to add the two events shown below.

Level 1 (scene) → On Init

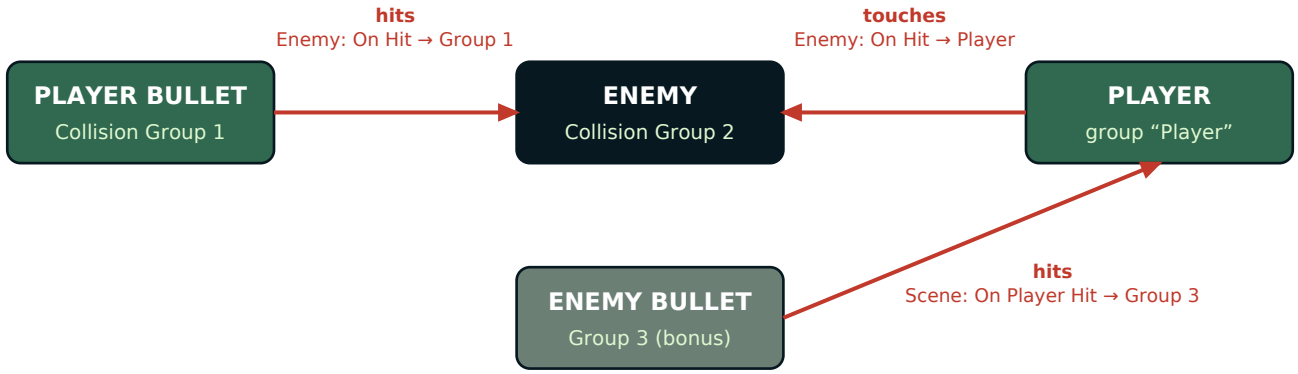
Attach Script To Button
Button: **A** · Override default button action: **on**

  Launch Projectile
  Sprite Sheet: **bullet**
  Source: **Player** · Offset X: **0** · Offset Y: **0**
  Direction: **Up** (a fixed direction, not the actor’s direction)
  Speed: one of the **fastest** options · Life Time: **1** second
  Destroy On Hit: **on**
  Collision Group: **1** · Collide With: **2** (only group 2)

  Play Sound Effect
  Effect: **Beep**, short and high (or **Tone**)

Collision groups – the most important idea in this step

How does a laser know that it hit an alien? With **collision groups**. Think of them as coloured team shirts. We decide:



Our team plan. Group 1 = your lasers, Group 2 = aliens, Group 3 = alien bullets (only in Bonus A).

- Our laser wears shirt **1** (“Collision Group: 1”).
- It only checks for things wearing shirt **2** (“Collide With: 2”). Aliens will wear shirt 2.
- When a group-1 laser hits an alien, the alien’s **On Hit → Group 1** script runs. We will write that script in Step 8.

WHY DO WE DO THIS?

Life Time 1 second: the laser disappears after a second, long before anyone would notice. This keeps the number of objects low, which keeps the Game Boy fast.

Destroy On Hit: one laser kills one alien, then vanishes – it doesn’t fly through a whole row.

CHECKPOINT – TEST IT!

Press ▶ **Play**, start the game, and press **Z** (A). Lasers should fly up from your ship with a beep. Tap A quickly to fire many shots. (They pass through nothing yet – there are no aliens.)

TIP

Want the B button to shoot too? Add a second **Attach Script To Button** for button **B** with the same two events inside. Many events can be copied with their menu → Copy, and pasted.

Step 7 – The Game Over scene

Before we create aliens, we create the place you go when you run out of lives. (The aliens’ scripts need to point at it.)

1. Click **+** → **Scene** and place it somewhere free, e.g. below the Title.
2. Name: **Game Over**. Type: **Top Down 2D**. Background: **gameover**.
3. Select the scene and open **On Init**. Build the script below.

Game Over (scene) → On Init

Hide Actor
Actor: **Player**

Play Sound Effect
Effect: **Crash**

Display Dialogue
Text (two lines):
Score: **$Score$**
Press A to retry

Change Scene
Scene: **Title** · any position · any direction

How to put the score into the text

1. Click into the Display Dialogue text box and type **Score:**
2. Now type a dollar sign **$**. A small list of variables pops up.
3. Choose **Score**. A special token for the variable is inserted. When the game runs, it is replaced by the actual number.
4. Press Enter for a new line and type **Press A to retry**.

WHY DO WE DO THIS?

Hide Actor: Player – a Top Down 2D scene always contains a player. We don’t want a ship sitting on the Game Over screen, so we hide it immediately.

Display Dialogue automatically waits for the player to press A before the script continues. So the player has time to read the score; then Change Scene takes them back to the title.

NOTE

A dialogue box can show only a few short lines (about 18 characters per line). Keep the text short.

Step 8 – Your first alien

8A. Place it

1. Zoom so you can see the **lower part** of Level 1.
2. Click **+** → **Actor** (shortcut: **A**) and click inside Level 1, a bit above where the player starts.
3. In the sidebar set: Name **Enemy 1**. Position **X 4**, **Y 140**. Sprite Sheet: **enemy**.
4. Set **Movement Speed** to a slow value (e.g. **1**, or “Speed 1”).
5. Set **Collision Group** to **2**. New script tabs for **On Hit** appear.

WHY DO WE DO THIS?
Aliens wear team shirt **2**, because our lasers are set to “Collide With: 2”. If you forget this step, lasers fly straight through aliens.

8B. Make it move – On Update

Remember: the camera is flying upwards, so an alien that just stays in place already seems to come down towards you. We only add a side-to-side wiggle to make it harder to hit.

Enemy 1 (actor) → On Update

Actor Move Relative
Actor: **Self** · X: **+3** · Y: **0** (tiles)

Actor Move Relative
Actor: **Self** · X: **-3** · Y: **0** (tiles)

WHY DO WE DO THIS?
On Update runs **again and again** while the alien is on screen: right 3 tiles, left 3 tiles, right 3, left 3… That loop is all you need for a moving enemy. We use **Self** instead of “Enemy 1” so the script still works after we copy the alien many times.

8C. When a laser hits it – On Hit → Group 1

Open the **On Hit** tab, then the **Group 1** sub-tab (hit by a group 1 projectile = our laser):

Enemy 1 (actor) → On Hit → Group 1 (hit by the player’s laser)

Actor Collisions Disable
Actor: **Self**

Hide Actor
Actor: **Self**

Play Sound Effect
Effect: **Crash**

Variable Increment By 1
Variable: **Score**

Deactivate Actor
Actor: **Self**

8D. When the ship crashes into it – On Hit → Player

Now open the **Player** sub-tab of On Hit:

Enemy 1 (actor) → On Hit → Player (touched the ship)

Actor Collisions Disable
Actor: **Self**

Hide Actor
Actor: **Self**

Play Sound Effect
Effect: **Crash**

Variable Decrement By 1
Variable: **Lives**

If Variable Compare With Value
Variable: **Lives** · Comparison: **Less Than Or Equal To (≤)** · Value: **0**

> **Change Scene (in the TRUE part of the If)**
> Scene: **Game Over** · any position

Deactivate Actor
Actor: **Self**

WHY DO WE DO THIS?

Why do we **first** disable collisions and hide the alien, and only remove it (Deactivate) at the **end**?

• **Collisions off first:** the game checks for collisions many times per second. Without this, one crash could be counted several times – and you would lose all 3 lives at once!

• **Hide first:** the alien vanishes instantly, so the explosion feels immediate.

• **Deactivate last:** Deactivate completely removes the actor and stops its scripts. If it came first, it could stop this very script before the score or lives are changed.

NOTE

Event names can be worded a little differently between versions – e.g. “Actor Collisions Disable” may appear as “Set Actor Collisions Disable”; “Variable Increment By 1” may be called “Increment Variable”. Type a keyword (*collision*, *increment*, *deactivate*, *compare*) in the event search box and pick the matching one.

CHECKPOINT – TEST IT!

Press ▶ **Play**. Soon after starting, the alien appears at the top of the screen and wiggles left and right as it comes down.

Shoot it: it should disappear with a crash. **Next game:** fly into it: crash sound, and it disappears. (You have 3 lives, so no Game Over yet.)

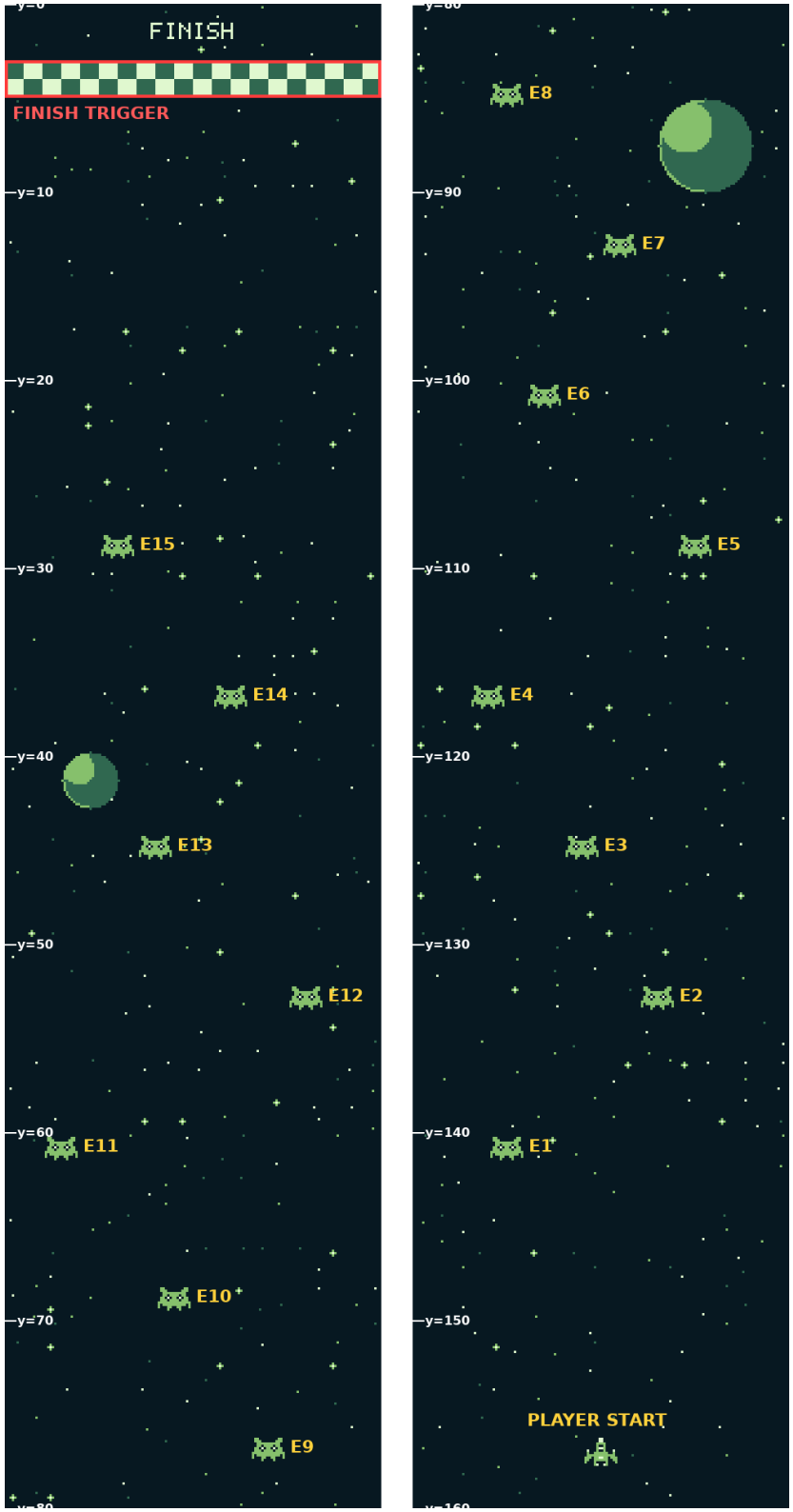
If lasers pass through it, check: Collision Group of the alien is 2, the laser has Collide With 2, and the script is in the **Group 1** sub-tab.

Step 9 – Fill the sky with aliens

One alien is not a game. Let’s make 15! Because every script uses **Self**, we can simply copy the first alien.

1. Click **Enemy 1** in the Game World to select it.
2. Copy it: **Ctrl+C** (Cmd+C on Mac), or use the actor’s menu in the sidebar (▾/ ⋮) → **Copy**.
3. Paste: **Ctrl+V** (Cmd+V), then place the new alien in Level 1. A copy has all the same scripts and settings.
4. In the sidebar give it a name (**Enemy 2**…) and type the exact **X** and **Y** from the table below.
5. Repeat until you have **15 aliens**.

Alien	X	Y	Alien	X	Y	Alien	X	Y
Enemy 1	4	140	Enemy 6	6	100	Enemy 11	2	60
Enemy 2	12	132	Enemy 7	10	92	Enemy 12	15	52
Enemy 3	8	124	Enemy 8	4	84	Enemy 13	7	44
Enemy 4	3	116	Enemy 9	13	76	Enemy 14	11	36
Enemy 5	14	108	Enemy 10	8	68	Enemy 15	5	28



The whole Level 1 map, cut into two halves: left = top half (y 0–79), right = bottom half (y 80–159). E1 is the first alien you meet, E15 the last. The red rectangle is where the FINISH trigger goes in Step 10.

WHY DO WE DO THIS?

Why 8 tiles between aliens? The screen is 18 tiles tall, so at most 2–3 aliens are on screen at once. That respects the “about 10 actors per screen” limit, and it gives the player time to react.

Why no alien with X bigger than 15? Each alien moves 3 tiles to the right and is 2 tiles wide: 15 + 3 + 2 = 20 = the right edge of the level.

TIP

Pro tip – Prefabs. GB Studio 4 can turn an actor into a **prefab** (a reusable template; look in the actor’s sidebar menu or the Prefabs section of the Navigator). Every alien placed from the prefab shares its scripts, so if you later change the alien’s behaviour, you change it once and all 15 update. For your first game, copy & paste is perfectly fine.

CHECKPOINT – TEST IT!

Play the whole level. Aliens should keep coming, wiggling as they fall. Shoot as many as you can. Try crashing into three of them on purpose: after the third crash you should see the **GAME OVER** screen with your score. Press A – you’re back at the title. Start again – lives are reset to 3.

Step 10 – The Victory scene and the finish line

10A. The Victory scene

1. Click **+** → **Scene** and place it somewhere free.
2. Name: **Victory**. Type: **Top Down 2D**. Background: **victory**.
3. Build its On Init script:

Victory (scene) → On Init

Hide Actor
Actor: **Player**

Play Sound Effect
Effect: **Tone** (a happy sound)

Display Dialogue
Score: **$Score$**
Lives left: **$Lives$**

Display Dialogue
Thanks for playing!
Press A

Change Scene
Scene: **Title**

10B. The finish line trigger

1. Zoom in on the **top** of Level 1, where the checkered FINISH line is drawn.
2. Click **+** → **Trigger** (shortcut: **T**).
3. Click and **drag** across the checkered band to draw a rectangle covering the full width of the level (as in the map in Step 9). If the sidebar shows position and size fields, use **X 0**, **Y 3**, **Width 20**, **Height 2**.
4. Name the trigger **Finish** and build its **On Enter** script:

Finish (trigger) → On Enter

Change Scene
Scene: **Victory** · any position

WHY DO WE DO THIS?

Triggers are invisible. The *picture* of the checkered line is part of the background; the trigger is the invisible “sensor” lying on top of it. When the ship flies into the sensor, the script runs. At the top of the level the scrolling stops, and because the ship has free movement, the player can fly up into the line.

CHECKPOINT – TEST IT!

Play the complete game from start to finish. Survive to the top and fly into the FINISH line: the **MISSION COMPLETE** screen should appear with your score and remaining lives.

Congratulations – you have a complete, working game!

Step 11 – Sound and music

You already have sound effects. Here is how to add background music.

1. Get a music file. GB Studio uses **.uge** files (made in GB Studio’s own built-in Music Editor or the hUGETracker program) or **.mod** files, depending on the music driver chosen in Settings. The official site (**gbstudio.dev**, Music page) links to free tracks you can use.
2. Put the music file in your project’s **assets/music** folder (just like the art in Step 1).
3. Select **Level 1**, open **On Init**, and add the event **Play Music Track** (search: *music*). Choose your song. Drag this event to the **top** of the script, above Attach Script To Button.
4. If you like, add another Play Music Track to the Title scene’s On Init (as the first event), and a Stop Music event in Game Over.

WATCH OUT
On the Game Boy, music and sound effects share the same 4 sound channels. A sound effect may briefly interrupt one channel of the music. That’s normal and part of the retro charm.

Step 12 – The final test

Before sharing your game, play it several times and check every item. This is what professional testers do!

✓	Test	Expected result
☐	Start the game	Title screen appears; Start/A goes to Level 1
☐	Level scrolls	Stars move down, the ship starts at the bottom
☐	Ship moves	Arrow keys move the ship freely around the screen
☐	Shooting	A launches a laser straight up, with a sound
☐	Hit an alien	It vanishes, crash sound, score goes up
☐	Crash into an alien	It vanishes, you lose exactly ONE life
☐	Lose 3 lives	Game Over screen shows the correct score
☐	Retry	Back at the title; new game starts with 3 lives and 0 score
☐	Reach the top	FINISH line → Mission Complete with score and lives
☐	Difficulty	Too hard? Lower the scroll speed or remove an alien. Too easy? Raise it.

Step 13 – Export and share your game

Your game is done. Let’s turn it into a real Game Boy file.

1. In the menu bar, open **Game → Export As** (in some versions: **Build** menu or the build/export button).
2. Choose **ROM**. GB Studio builds a file called something like **StarPatrol.gb** and shows you its folder (usually a *build* folder inside your project).
3. Open that file in any Game Boy emulator to play it – or copy it to a flash cartridge to play on a real Game Boy!
4. Choose **Export As → Web** to get a folder with an HTML version. Zip that folder and you can upload it to **itch.io** as an HTML5 game, playable in any browser – even on phones, with on-screen buttons.

TIP
When you upload to itch.io, set the embed size to a multiple of 160×144 (for example 480×432 or 640×576) so the pixels stay sharp.

CHECKPOINT – TEST IT!
Send the file or link to a friend. Watching someone else play your game is the best way to learn what to improve next.

PART 3

Make it yours

Bonus features, troubleshooting, and a cheat sheet with every script.

Bonus A – Aliens that shoot back

Real shoot-’em-ups have enemy bullets. This uses the **enemy_bullet** sprite and collision group **3**.

A1. Make an alien shoot

Pick a few aliens (not all – that would be too hard and could slow the game). Change their **On Update** script to:

Enemy (actor) → On Update · shooting version

Launch Projectile
Sprite Sheet: **enemy_bullet** · Source: **Self**
Direction: **Down** · Speed: a **slow/medium** option · Life Time: **2**
Destroy On Hit: **on** · Collision Group: **3** · Collide With: **Player** only

Actor Move Relative
Self · X **+3** · Y 0

Actor Move Relative
Self · X **-3** · Y 0

A2. Make the player get hurt

Enemy bullets don’t hit an alien – they hit the **player**. The player’s reaction lives in the **scene**: select Level 1, find the **On Player Hit** script tab, and choose the **Group 3** sub-tab:

Level 1 (scene) → On Player Hit → Group 3

Play Sound Effect
Effect: **Crash**

Variable Decrement By 1
Variable: **Lives**

If Variable Compare With Value
Lives ≤ 0

  Change Scene (TRUE part)
  Scene: **Game Over**

CHECKPOINT – TEST IT!
Play. Shooting aliens fire a ball down every time their movement loop starts. Getting hit costs one life. If the game feels unfair, make those bullets slower or give only 3–4 aliens the ability to shoot.

WATCH OUT

Each projectile uses sprite memory and processing time. If the game starts to slow down or sprites flicker, reduce the number of shooting aliens or shorten the bullets’ Life Time.

Bonus B – More ideas to try

Diving aliens

Give some aliens a different On Update: **Move Relative X 0, Y +6** then **Wait 1 second**. They will rush down towards you. Mix wiggling and diving aliens for variety.

Two-hit tough alien

Use a second variable (for example, an actor-local variable if your version offers it, or a global like **BossHP**). In On Hit → Group 1, subtract 1 and only remove the alien when it reaches 0.

Faster level 2

Duplicate Level 1 (select it, copy/paste), name it Level 2, rearrange the aliens, and change the Finish trigger of Level 1 to go to Level 2 (facing **Up**, of course!) instead of Victory.

Explosion animation

In the Sprites view you can give the enemy sprite a second **animation state** (for example called *Explode*) with a few frames. In On Hit, instead of hiding immediately, use **Set Actor Animation State: Explode**, wait a moment, then hide and deactivate it.

Score on screen

Showing a live score while playing uses the **overlay** (the window layer) and text-drawing events. It’s a great next challenge once you are comfortable – search the GB Studio docs for “overlay”.

Difficulty settings

The Shoot ’Em Up scroll speed in Settings changes the pace of the whole game. Slower = more relaxed; faster = arcade intensity.

Troubleshooting

Problem	Most likely cause	Fix
The game starts in the wrong scene	Starting scene not set	Click empty Game World area → project settings → Starting Scene: Title (Step 4B)
The level does not scroll, or scrolls sideways	Wrong direction or wrong scene type	Title’s Change Scene must use Direction **Up**; Level 1 Type must be **Shoot ’Em Up**
The ship can only move along one line	Movement locked to one edge	Settings → Shoot ’Em Up → choose free movement
Pressing A does nothing	Script in wrong place	Attach Script To Button must be in **Level 1 scene → On Init**, with Launch Projectile *inside* it
Lasers fly through aliens	Group mismatch	Laser: Group 1, Collide With 2. Alien: Collision Group 2. Script in On Hit → **Group 1**. Check collision boxes (Step 2)
One crash costs several lives	Collisions still on	Make **Actor Collisions Disable: Self** the first event in On Hit
Score/lives don’t change	Variable not selected	Open the event and choose Score/Lives in the variable dropdown
Second game starts with 0 lives	No reset	Title On Init must set Score = 0 and Lives = 3 *before* Change Scene
Aliens flicker or vanish	Too many actors on one screen	Keep 10 or fewer actors per screen area; spread them out
“Too many tiles” warning	Background too detailed	Use the Logo type (title) or simplify the image
Art looks wrong / colours changed	Not using the exact palette	Use only the colours from Chapter 5
Finish line never triggers	Trigger not over the line	Trigger at X 0, Y 3, width 20, height 2; ship needs free movement
Dialogue shows “$Score$” as text	Typed by hand	Delete it and insert the variable by typing **$** and picking Score from the list
Player visible on Game Over screen	Missing Hide Actor	First event: Hide Actor → Player

TIP

Still stuck? GB Studio has a friendly community: the official **Discord** and the **r/gbstudio** subreddit (links at gbstudio.dev). GB Studio also has a built-in **Debugger** that shows variables while the game runs – very useful to see whether Score and Lives really change.

Cheat sheet – every script in STAR PATROL

Use this page to double-check your project. It lists every script in the finished game (without the bonus).

Where	Script tab	Events in order
Title scene (Logo, bg: title)	On Init	Set Score = 0 → Set Lives = 3 → Await Input (Start, A) → Play Sound (Beep) → Change Scene: Level 1, X 9, Y 156, **Direction Up**
Level 1 scene (Shoot ’Em Up, bg: level1)	On Init	[Play Music Track] → Attach Script To Button A { Launch Projectile: bullet, from Player, Up, fast, life 1 s, destroy on hit, Group 1, collide with 2 → Play Sound (Beep) }
Enemy 1–15 actors (sprite: enemy, Collision Group 2)	On Update	Move Relative Self +3,0 → Move Relative Self −3,0
	On Hit → Group 1	Collisions Disable Self → Hide Self → Sound Crash → Score +1 → Deactivate Self
	On Hit → Player	Collisions Disable Self → Hide Self → Sound Crash → Lives −1 → If Lives ≤ 0 { Change Scene: Game Over } → Deactivate Self
Finish trigger (X 0, Y 3, 20×2)	On Enter	Change Scene: Victory
Game Over scene (Top Down 2D, bg: gameover)	On Init	Hide Player → Sound Crash → Dialogue “Score: $Score$ / Press A to retry” → Change Scene: Title
Victory scene (Top Down 2D, bg: victory)	On Init	Hide Player → Sound Tone → Dialogue “Score / Lives left” → Dialogue “Thanks for playing!” → Change Scene: Title

Settings checklist

- Shoot ’Em Up player sprite = **player_ship**; movement = **free**; scroll speed = to taste.
- Top Down 2D player sprite = anything (it’s hidden), e.g. **player_ship**.
- Sprite collision boxes adjusted (Step 2).
- Variables: **Score**, **Lives**.

Glossary

Term	Meaning
Actor	Any object placed in a scene that can have scripts (enemies, characters, items).
Asset	A file used by the game: picture, sound, music.
Background	The (non-moving) picture of a scene. Must be 4 colours and a multiple of 8 pixels in size.
Bounding / collision box	Invisible rectangle used to detect touching.
Collision group	A “team” (Player, 1, 2, 3) that decides which things can hit each other and which On Hit script runs.
Deactivate	Completely switch off an actor: invisible, no collisions, scripts stopped.
Emulator	A program that pretends to be a Game Boy so you can play .gb files on a computer or phone.
Event	A single instruction in a script.
Hitbox	Another name for collision box.
Projectile	A temporary flying sprite launched by a script.
ROM	The finished game file (.gb) that a Game Boy or emulator runs.
Scene	One area/screen of the game.
Script	A list of events run from top to bottom.
Self	In an actor’s script: “this actor, whichever one it is”. Makes copied actors work.
Shmup	Short for shoot-’em-up.
Sprite	A small (16×16) moving picture.
Tile	An 8×8 pixel square; everything on the Game Boy is built from tiles.
Trigger	An invisible area that runs a script when the player enters or leaves it.
Variable	A named number the game remembers, like Score.

TIP

Where to go next: the official documentation at **gbstudio.dev/docs**, the free community art collection “GB Studio Community Assets” on GitHub, and game jams on itch.io (search “GB Studio jam”) where you make a small game in a few days with other beginners.

Most of all: make another small game. Then another. Every game you finish teaches you more than any guide. **Have fun!**